
Bridging RTL and Theorem Proving: LLM-Assisted Translation from Verilog to Lean

SooHyuk Cho

Department of Electrical and Computer Engineering
Princeton University
Princeton, NJ
soohyuk.cho@princeton.edu

Yu-Wei Fan

Department of Electrical and Computer Engineering
Princeton University
Princeton, NJ
yf9172@princeton.edu

Abstract

Interactive theorem proving offers an appealing path for hardware verification, but its practical use for RTL remains limited by the manual effort required to formalize designs in a prover-native language. This paper studies *Verilog-to-Lean autoformalization*: the problem of translating Verilog RTL modules into structured Lean 4 models that can serve as a basis for later proof-oriented reasoning. We define fixed Lean target schemas for combinational, sequential, and controller-style modules, and evaluate schema-constrained LLM generation on a benchmark of 38 Verilog designs spanning four tiers of complexity. Our pipeline combines prompt-based generation with Lean compilation, schema checking, sorry-free checking, and bounded iterative repair. In one-shot generation, both GPT-5.4 and Gemini 2.5 Pro produce schema-conformant outputs for all 38 modules and compile 28 of 38. Under checker-guided refinement, GPT-5.4 reaches 38 of 38 compilable artifacts, while Gemini 2.5 Pro reaches 36 of 38. The remaining failures are concentrated in harder Tier 4 designs and suggest that the main difficulty is not recovering the broad RTL structure, but producing Lean artifacts that satisfy executable typing, API, and theorem-shape constraints. Although our current evaluation does not establish full semantic equivalence between generated Lean models and their source RTL, the results show that schema-constrained, checker-guided autoformalization is already a practical and effective first step toward Lean-based hardware verification. Full code is available at <https://github.com/soohyuk-cho/COS598B-Final-Project-Verilog-To-Lean.git>.

1 Introduction

Pre-silicon verification of hardware computing chips remains a central challenge in modern hardware design. RTL bugs that survive to fabrication are costly, and simulation-based verification explores only a limited portion of the design state space. Formal methods help address this gap. In practice, hardware formal verification is still dominated by techniques like model checking and equivalence checking [1]. Interactive theorem proving (ITP) remains far less common in mainstream RTL workflows. The appeal of interactive theorem prover is clear: it supports richer specifications, reusable abstractions, and machine-checked proofs. Yet this flexibility comes at a price, since the design and its properties must first be expressed in the language of the prover.

Lean [2] is an interactive theorem prover based on dependent type theory that has become popular for building machine-checked proofs and verifiably correct software, supported by a growing ecosystem through Lean 4 [2]. Its surrounding libraries such as mathlib [3] and CSlib [4] make Lean increasingly attractive as a foundation for verified reasoning in mathematics and computer science thanks to its strong automation via tactics and integration with external solvers. In the LLM era, Lean offers another important advantage: it can act as a precise checker for machine-generated formal artifacts. A model may generate definitions, proof sketches, or entire proofs, but Lean accepts them only when they typecheck. Recent work on Lean-centered theorem proving and proof assistance, including LeanDojo [5], Lean Copilot [6], and AlphaProof [7], further suggests that LLMs and formal proof systems can be combined in a productive way.

Despite this promise, Lean is not yet a standard environment for hardware verification. RTL designers write Verilog or SystemVerilog, and would need to manually build faithful Lean model of hardware modules, which is tedious and difficult to scale. More broadly, prior proof-assistant-based hardware verification frameworks such as Kami [8] and Silver Oak [9] show that theorem proving can support serious hardware-level reasoning, but they also highlight the amount of formalization effort typically required before proofs can even begin. For Lean-based hardware verification, the immediate bottleneck is therefore not proof search alone, but formalization: getting RTL into Lean in a structured, faithful, and checkable form.

This project studies the bottleneck of autoformalization from RTL to a Lean model directly. We focus on the problem of translating Verilog RTL into Lean 4 models using LLMs under fixed target schemas. For combinatorial designs, the target is a pure function over typed input and output structures. For sequential designs, it is an explicit state-machine model with initialization, step, and output functions. Finally, for controller-style modules, it extends to richer state and stronger behavioral invariants. We evaluate this setting on a benchmark of 38 Verilog modules spanning combinational, sequential, and controller-oriented designs, and study how far schema constraints, compilation checks, and checker-guided refinement can improve the reliability of LLM-generated Lean artifacts for the hardware.

In summary, our contributions through this project are:

- We define fixed Lean target schemas for combinational, sequential, and controller-style Verilog modules.
- We manually curate a 38-module Verilog benchmark across four different tiers based on the design complexity.
- We evaluate multiple LLM-based translation settings, from one-shot generation to checker-guided refinement.
- We analyze compilation, schema conformance, and common failure modes such as invalid Lean API use, structural mismatches, and unsound proof placeholders.

2 Background and Problem Formulation

2.1 Hardware Verification and Theorem Proving

Formal Verification is a central part of pre-silicon hardware validation, where the goal is to detect design errors before fabrication. In mainstream RTL practice, this is largely filled by simulation, model checking, and equivalence checking. These techniques are highly effective and admit automation for many finite-state hardware problems and remain the dominant practical workflow for industrial-level verification [1]. However, these automated hardware formal tools are less natural when the goal is to state richer functional specifications, reason compositionally across abstraction levels, or produce reusable proof artifacts. Also, humans often rely on formal-method experts to manually perform design abstraction and proof decomposition for system-level verification.

ITP offers a complementary approach to this. Instead of expressing correctness only through automated verification techniques over a fixed property language, ITP allows the design semantics and the proof obligations themselves to be written explicitly inside a logic. This flexibility has supported important hardware verification efforts over multiple generations of proof assistants. These systems demonstrate that theorem proving is viable for hardware, but they also make clear that formalization itself is a major part of the effort.

2.2 Why Lean as a Target Language

We use Lean 4 as the target language because it can naturally express the kinds of structures that arise in RTL modeling: typed module interfaces, explicit state, pure combinational functions, and step-wise transition systems [2]. Its support for executable definitions and bit-vector-level reasoning makes it suitable for representing hardware behavior in a programmable and checkable format. Lean provides a small trusted kernel, so generated artifacts are not accepted on the basis of plausibility, but only after passing a precise logical check. For this project, these properties make Lean a useful target for studying whether RTL can be translated into a machine-checked formal representation.

2.3 Problem Definition

We study the problem of *Verilog-to-Lean autoformalization*. Given a Verilog RTL module, the goal is to generate a Lean artifact that represents the behavior of the module under a fixed target schema. The output is not arbitrary Lean code, but a structured formal model that must satisfy several verification-oriented requirements: 1) it should conform to the intended schema, compile successfully, avoid unsound placeholders such as `sorry`, and support representative checks of the expected behavior.

2.4 Scope of This Work

Our focus is the formalization step rather than end-to-end proof automation. We do not attempt to prove full functional correctness for realistic SoC-scale designs, nor do we aim to replace existing industrial RTL verification flows. Instead, we ask whether LLMs can reliably translate RTL into Lean in a form that is structured enough to support later proof-oriented reasoning.

This focus also leaves open more structured hybrid directions. Existing front ends such as Yosys and Surelog/UHDM already provide parser- and IR-based access to RTL syntax and elaborated designs [10, 11]. Such infrastructure could eventually be combined with LLM-based generation, but in this work we concentrate on the direct auto-formalization problem and evaluate how far schema constraints and checker feedback can improve the quality of generated Lean models.

3 Lean Target Schema

A central design choice for this project is to translate Verilog modules into *fixed Lean Target Schemas* rather than allowing unconstrained code generation. This choice serves two purposes. First, it reduces ambiguity in the translation task by defining in advance what a valid Lean representation should look like for each class of hardware module. Second, it makes evaluation more systematic. Once the target structure is fixed, we can check not only whether a generated artifact compiles, but also whether it conforms to the intended interface and modeling discipline.

Our schemas are organized by **module type**. This reflects the fact that different classes of RTL designs require different modeling patterns. A small combinational block can often be represented as a pure function, while a sequential design requires explicit state and a cycle-level transition function. Controller-style modules, such as FIFOs or arbiters, typically require richer state and stronger behavioral properties. By separating these cases, we aim to preserve a consistent formalization style without forcing all modules into a single overly general representation.

The full schema for each module type is provided in Appendix A.1.

3.1 Schema A: Combinatorial Modules

For combinational modules, the target representation is a pure Lean model with typed input and output structures together with an evaluation function: `eval : Inputs -> Outputs`. This is intended for modules whose behavior is determined entirely by their current inputs, with no persistent internal state. Typical examples include multiplexers, decoders, encoders, comparators, and simple ALUs.

The main advantage of this schema is that it keeps the generated Lean artifact simple, explicit, and easy to read. This module interface is represented directly through typed fields, and the functional

behavior is captured in a single executable definition. This makes the translation easy to inspect and also supports lightweight theorem statements over representative input cases. In practice, Schema A serves as the simplest setting for evaluating whether an LLM can correctly map RTL operators, bit widths, and conditional logic into Lean.

3.2 Schema B: Sequential Modules

For sequential modules, a pure input-output function is no longer sufficient because the design behavior depends on persistent state. We therefore use a state-machine schema with the following components:

- `State`
- `Input`
- `Output`
- `init`
- `step`
- `out`

Here, `init` defines the initial state, `step` models one cycle of state transition, and `out` exposes the observable outputs associated with a given state and input. This schema is intended to capture common RTL patterns such as counters, shift registers, accumulators, and simple finite-state machines.

The main purpose of Schema B is to make clocked behavior explicit. Rather than embedding sequential semantics implicitly over the generated code, the schema forces the translation to represent the reset logic, enable conditions, update priority, and next-state behavior in a structured form. This makes the resulting Lean model easier to check and compare across modules, and it provides a natural basis for later theorem statements about cycle-by-cycle behavior.

3.3 Schema C: Controller-Style Modules

Some designs require a richer representation than basic sequential state machines. Controller-style modules, including FIFOs, arbiters, and related control logic, often involve structured internal state, auxiliary storage, or stronger behavioral expectations such as ordering, mutual exclusion, or correctness of control flow. For these cases, we use an extended schema that builds on Schema B while allowing more expressive state definitions and stronger property skeletons.

The purpose of Schema C is not to introduce a completely different modeling framework, but to extend the same general discipline to more complex RTL blocks. In particular, we preserve the explicit state-transition structure while allowing the state to include buffers, pointers, or named controller states when needed. This gives the translation enough flexibility to represent realistic controller logic without losing the structural regularity needed for compilation checks and schema conformance tests.

3.4 Why Fixed Schemas Matter

These schemas play an important role in both generation and evaluation. On the generation side, they provide the LLM with a precise target format, reducing the freedom to produce stylistically inconsistent or semantically underspecified Lean code. On the evaluation side, they allow us to distinguish different types of success and failure: a generated artifact may compile but violate the intended schema, or it may match the schema while still failing on semantic checks. This separation is important for understanding where the translation process succeeds and where it breaks down.

More broadly, the schemas turn Verilog-to-Lean translation into a structured autoformalization problem. Instead of asking the model to produce arbitrary formal code, we ask it to instantiate one of a small number of well-defined formal patterns. This design is central to the project, since it makes checker-guided refinement, structural validation, and benchmark-wide comparison substantially more meaningful.

4 Benchmark

To evaluate Verilog-to-Lean autoformalization in a controlled setting, we manually curated a benchmark of **38 Verilog modules** spanning **four completed tiers of design complexity**. The benchmark is intentionally structured rather than purely collection-based: lower tiers emphasize direct functional translation, while higher tiers introduce state, control structure, and richer behavioral requirements. This staged design allows us to study not only overall translation quality, but also how performance changes as the semantic complexity of the source RTL increases.

4.1 Benchmark Tiers

Tier 1: Simple combinational modules. This tier contains small stateless designs such as multiplexers, decoders, encoders, comparators, and simple ALUs. These modules serve as the most direct test of whether an LLM can translate Verilog expressions, bit-level operators, and conditional logic into the target Lean schema. Because they do not require persistent state, they are also the cleanest setting for isolating syntax and interface-translation errors.

Tier 2: Sequential modules. The second tier contains registered designs such as counters, shift registers, accumulators, and simple finite-state machines. These modules require the translation to model explicit state, reset behavior, and cycle-level updates. As a result, they test whether the generated Lean artifact correctly instantiates the sequential schema rather than merely producing a syntactically valid functional encoding.

Tier 3: Controller-style modules. The third tier contains more complex designs such as FIFOs, arbiters, and related controller logic. These modules introduce richer internal structure, including control state, auxiliary storage, and behavioral expectations such as ordering or mutual exclusion. This tier is therefore the most demanding part of the original benchmark and serves as a practical stress test for the robustness of the translation framework.

Tier 4: Property-oriented and Complex Datapath Modules. The fourth tier extends the benchmark toward more challenging datapath- and microarchitecture-oriented designs. Our current Tier 4 set includes arithmetic units such as dividers or FPUs, as well as a reorder-buffer-style design. Compared with the earlier tiers, these modules are less amenable to simple local functional summaries and place greater pressure on the translation system to recover structured control, multi-step datapath behavior, and richer semantic intent. Tier 4 is important because it begins to move the benchmark beyond small educational RTL blocks toward modules whose correctness is typically discussed in terms of higher-level behavioral properties. For arithmetic units, the challenge lies in faithfully representing nontrivial datapath structure and bit-level behavior in Lean. For modules such as `rob.v`, the challenge is closer to property-oriented modeling, where the relevant correctness conditions involve ordering, state consistency, and control discipline rather than only direct input-output functionality. Note that some Tier 4 designs were sourced from externally available implementations, including public GitHub repositories accompanied by testbenches, whereas the designs in the earlier tiers were written entirely by us.

4.2 Current Scope

The current benchmark spans four completed tiers and forms the basis of the evaluation in this report. We center the evaluation on the 38 curated modules for which the schema definitions, prompt templates, checker infrastructure, and reference Lean artifacts are already in place.

5 Methods

Our method treats Verilog-to-Lean translation as a constrained generation problem. Given a Verilog module, we first assign it to one of the target schema classes described in the previous section. The language model is then prompted to generate a Lean artifact that instantiates the corresponding schema. The generated output is passed through automated checks, including Lean compilation and structural validation, and may optionally be revised through a bounded repair loop. This design lets us evaluate not only whether a model can produce plausible Lean code, but whether it can produce Lean code that is structurally consistent and machine-checkable.

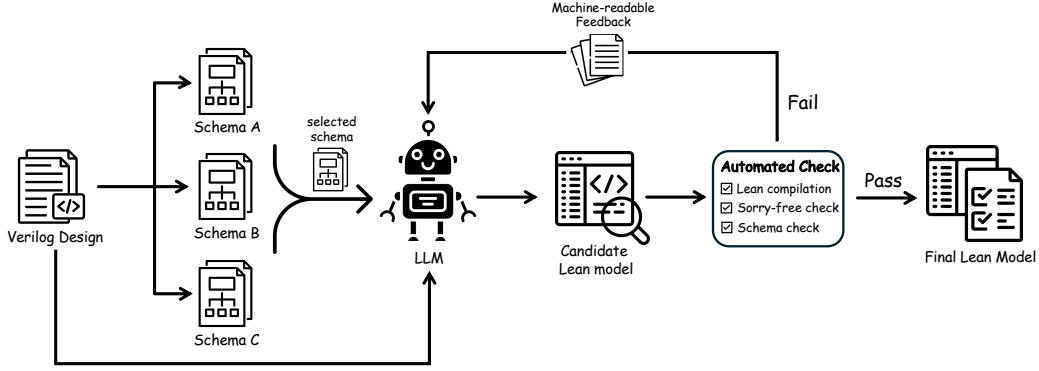


Figure 1: Translation pipeline for Verilog-to-Lean auto-formalization.

5.1 Translation Pipeline

The overall pipeline consists of four stages:

1. **Schema selection.** Each Verilog module is mapped to the appropriate target schema based on its design class.
2. **Initial generation.** The LLM receives the Verilog source together with schema-specific instructions and produces a Lean artifact.
3. **Automated checking.** The generated artifact is evaluated using Lean compilation and additional structural checks.
4. **Repair, if enabled.** When generation fails, the model receives machine-readable feedback and produces a revised Lean artifact, subject to a fixed iteration budget.

This pipeline separates generation from acceptance. The LLM is responsible for proposing a formalization, but the artifact is only counted as successful if it passes the downstream checks.

5.2 Prompting and System Variants

We evaluate several generation settings that differ in how much feedback is provided to the model.

Sys A: One-shot generation. The model receives the Verilog module and the target schema and generates a Lean artifact in a single pass. This is the simplest setting and serves as the baseline.

Sys B: Compilation-feedback generation. In this setting, the model is allowed to revise its output using Lean compilation success or failure as feedback. The goal is to test whether even a coarse checker signal improves reliability relative to one-shot generation.

Sys C: Checker-guided generation. Beyond compilation, this setting provides structural feedback derived from schema conformance and related checks. This allows the model to correct not only surface syntax issues, but also mismatches between the generated artifact and the intended modeling pattern.

Sys D: Iterative repair. The final setting allows a bounded number of repair rounds, typically one to three iterations, in which the model revises its previous output based on accumulated checker feedback. This setting tests whether repeated refinement can recover from earlier translation errors without relaxing the target schema.

Together, these systems define the feedback hierarchy used in our translation framework, ranging from direct one-shot generation to checker-guided revision. In the full pipeline, Systems A, B, and C correspond to increasingly strong feedback available during initial generation, while System D applies bounded iterative repair to modules that fail the initial compile check.

In the final empirical study reported here, we center the comparison on two settings: Sys A as the one-shot baseline and Sys D as iterative repair applied only to one-shot failures. Systems B and C are still part of the improved checker design and motivate the structure of the feedback pipeline, but they are not reported as separate benchmark-wide result tables in this final report. The prompts used for one-shot generation and iterative refinement are included in the Appendix A.2.

5.3 Role of Human Design

Although the generation itself is automated, the method relies on fixed human-designed scaffolding. In particular, we manually define the target schemas, prepare the prompt templates, set the repair budget, and curate the benchmark and reference artifacts. This separation is deliberate. The purpose of the study is not to eliminate all human design effort, but to test whether LLMs can automate the formalization step once the target representation and evaluation protocol are fixed.

5.4 Implementation Scope

Our experiments are centered on Lean 4 and a checker pipeline built around compilation and structural validation. Our primary model is GPT-5.4, which serves as the main generator throughout the current study. We also treat Gemini 2.5 Pro as a secondary model for limited cross-provider comparison when additional results are available. On the checking side, the generated artifacts target Lean 4.29, and compilation is performed through the standard Lean toolchain with Lake build support. In addition, we use a lightweight checker layer for schema-oriented structural validation. The emphasis of the method is, therefore, not on comparing too many models but on understanding how schema constraints and checker feedback affect the quality of generated Lean formalizations.

6 Evaluations

This section defines how we evaluate Verilog-to-Lean autoformalization once the translation pipeline and system variants have been fixed. Our goal is not merely to measure whether a model can produce syntactically plausible Lean code, but whether the resulting artifacts are usable as formal representations of RTL modules. For that reason, the evaluation emphasizes machine-checkable outcomes such as compilation, schema conformance, and the absence of unsound proof placeholders.

6.1 Evaluation Scope

Our evaluation is performed over the benchmark introduced in the previous section. The benchmark is organized by design tier so that results can be analyzed as a function of RTL complexity rather than only in aggregate.

The full framework defines multiple system variants, ranging from one-shot generation to checker-guided repair. In the final empirical results reported here, however, we focus on two concrete evaluation settings. First, we evaluate *Sys A one-shot generation* for GPT-5.4 and Gemini 2.5 Pro across all 38 benchmark modules. Second, we evaluate *Sys D iterative repair*, which is applied only to modules that fail compilation under the one-shot baseline. This design reflects the practical role of repair in our pipeline: it is a targeted recovery mechanism for failed initial translations rather than a separate first-pass generation mode.

Systems B and C remain part of the overall checker design and define intermediate levels of feedback strength, but they are not reported as separate benchmark-wide comparisons in the final report. Instead, their main contribution is reflected in the structure of the checker-guided workflow that supports the reported repair setting.

6.2 Metrics

We report the following metrics based on our checker.

Compilation success. A generated artifact is counted as compiling if it is accepted by the Lean toolchain under the target project configuration. This is the most basic validity check and filters out outputs with syntax or type errors.

Schema conformance. A generated artifact is counted as schema-correct if it follows the intended structural pattern for its assigned target schema. This metric is designed to distinguish genuine schema instantiation from arbitrary Lean code that happens to compile.

Sorry-free rate. We record whether the generated file contains `sorry`. This metric is important because `sorry` allows unfinished proofs to pass syntactically, making compilation alone an insufficient indicator of successful formalization.

Representative behavioral checks. When lightweight theorems or representative test cases are available, we additionally record whether the generated artifact supports those checks. This does not amount to full semantic equivalence with the original RTL, but it provides a stronger signal than compilation and schema conformance alone.

Repair improvement. For system variants that include feedback and revision, we measure whether repair improves the status of an artifact, such as turning a non-compiling output into a compiling one or recovering schema conformance after an initial failure. This metric is defined as part of the evaluation framework even when the present results focus primarily on one-shot generation.

These metrics are consistent with the checker signals used in the project implementation, in which compilation, `sorry`-freeness, and schema-level structural checks are treated as distinct indicators of quality.

7 Results

Tables 1 and 2 show the one-shot translation results for GPT-5.4 and Gemini 2.5 Pro respectively across all benchmark tiers. Tables 3 and 4 report cumulative compilation after iterative repair, and Table 5 tracks theorem count changes across repair iterations.

Tier	# Modules	Compiles	Sorry-free	Schema OK	Compile Rate
Tier 1	9	9/9	9/9	9/9	100.0%
Tier 2	8	8/8	8/8	8/8	100.0%
Tier 3	12	6/12	7/12	12/12	50.0%
Tier 4	9	5/9	9/9	9/9	66.6%
Total	38	28/38	33/38	38/38	73.7%

Table 1: Preliminary Lean target generation results for GPT-5.4-generated modules by tier.

Tier	# Modules	Compiles	Sorry-free	Schema OK	Compile Rate
Tier 1	9	9/9	9/9	9/9	100.0%
Tier 2	8	8/8	8/8	8/8	100.0%
Tier 3	12	9/12	12/12	12/12	75.0%
Tier 4	9	2/9	3/9	9/9	22.2%
Total	38	28/38	32/38	38/38	73.7%

Table 2: Preliminary Lean target generation results for Gemini Pro 2.5-generated modules by tier.

7.1 One-Shot Results

Across the four benchmark tiers, the generated Lean artifacts show a clear split between basic structural validity and full compilability. At the structural level, the results are uniformly strong: all 38 GPT-5.4-generated modules satisfy the schema checks, indicating that the translation process reliably produces outputs with the expected namespace, state/input and output structures, executable definitions, and theorem declarations. Compilation success, however, drops as the benchmark complexity increases. For GPT-5.4 generated artifacts, Tier 1 and Tier 2 compile completely (9/9 and 8/8, respectively), while Tier 3 compiles only 6/12 modules and Tier 4 compiles 5/9 modules. Overall, 28 of 38 modules compile successfully, for a total compile rate of 73.7%.

The `sorry`-free counts should be interpreted with some care. In this evaluation, `sorry`-freeness is determined not only by checking that the source text does not explicitly contain `sorry`, but also

by checking whether Lean emits a compiler warning such as “declaration uses sorry.” Under this criterion, the lower sorry-free rate in Tier 3 does not reflect intentional use of `sorry` placeholders in the generated files. Instead, the observed sorry-related warnings in Tier 3 are induced by other compilation failures, which then prevent Lean from validating the affected declarations cleanly. In other words, the main degradation in the harder tiers is driven by genuine compilation issues rather than by the model falling back to explicit proof holes. We sort the dominant classes of these failures in the next section.

Gemini 2.5 Pro produces comparable one-shot results at the structural level, with all 38 modules satisfying schema checks (Table 2). The compilation profile differs from GPT-5.4 in two notable ways. First, Tier 3 compiles 9/12 modules (vs. 6/12 for GPT-5.4), suggesting Gemini handles controller-style array operations more reliably in one-shot. Second, Tier 4 compiles only 2/9 modules (vs. 5/9 for GPT-5.4), with six FPU files also failing the sorry-free check—meaning Gemini fell back to explicit `sorry` placeholders rather than generating structurally incorrect proofs. This sorry pattern is distinct from the GPT-5.4 failures, where Tier 4 files were sorry-free despite not compiling, indicating the two models adopt different failure strategies under the same prompting constraints.

7.2 Common Failing Patterns

Although all generated artifacts in the current result set satisfy the high-level schema checks, several outputs still fail to compile. The failures are not uniformly distributed across the benchmark. They mainly appear in the controller-style and complex datapath tiers, where the generated Lean code must model arrays, memories, and more involved executable checks. We observed three recurring patterns.

First, several Tier 3 designs and one Tier 4 design fail because the model emits invalid or unsupported Lean array operations, most commonly involving `Array.mkArray` and `Array.get`. This pattern appears in `async_fifo_mem_buffer`, `async_shift_reg`, `circular_buffer_fifo`, `dual_port_fifo`, `threshold_buffer_fifo`, and `rob`. These errors are largely API-level failures: the generated model often has the intended memory-buffer structure, but uses array construction or indexing forms that do not typecheck under the target Lean environment.

Second, some generated models misuse `native_decide` for goals that still contain symbolic variables. For example, one generated theorem statement in `sync_fifo_shift_reg` quantifies over an input value `d : Bits8` and then tries to close the theorem using `native_decide`. Lean reports that the expected type must not contain free variables, because `native_decide` is intended for closed computational propositions rather than universally quantified symbolic equalities with parameters still present in the context. This failure also appears in the Tier 4 floating-point double-precision multiplier `fpu_dp_mul`. In these cases, the issue is not necessarily that the statement is false, but that the chosen proof tactic is inappropriate for the symbolic theorem shape.

Third, some failures arise from missing decidability instances for user-defined predicates. In `fpu_dp_div`, the generated transition function branches on predicates such as `isZeroVal s.b_e s.b_m`, but Lean cannot automatically synthesize a `Decidable (isZeroVal s.b_e s.b_m)` instance needed by the executable `if` branches in the transition function. Since executable `if` expressions over propositions require a decidability instance, predicates used in branch conditions must either be defined as Boolean functions or accompanied by explicit `Decidable` instances. This error is specific to more semantic encodings, such as floating-point classification, where the generated Lean model introduces helper predicates rather than only using primitive Boolean or bit-vector operations.

These failure modes suggest several straightforward fixes. The prompt and schemas can provide canonical templates for array construction and indexing, restrict the use of `native_decide` to closed goals, and include standard `Decidable` instances or Boolean versions for helper predicates used in executable branches. These changes would not require changing the overall translation pipeline, but would make the generated artifacts better aligned with Lean’s executable modeling discipline.

7.3 Iterative Refinement Results

GPT-5.4 fully recovers from all one-shot failures within two repair iterations (Table 3). The 10 non-compiling modules (6 from Tier 3 and 4 from Tier 4) are submitted to the repair loop. After Iter 1, 8 of the 10 compile; the two remaining Tier 3 modules (`sync_fifo_shift_reg`,

Tier	One-shot	+ Iter 1	+ Iter 2	Final Rate
Tier 1	9/9	9/9	9/9	100.0%
Tier 2	8/8	8/8	8/8	100.0%
Tier 3	6/12	10/12	12/12	100.0%
Tier 4	5/9	9/9	9/9	100.0%
Total	28/38 (73.7%)	36/38 (94.7%)	38/38 (100.0%)	100.0%

Table 3: Cumulative compilation results after iterative repair for GPT-5.4. Each column shows the total number of compiling modules including all previous iterations. Only modules that failed one-shot were submitted to repair.

Tier	One-shot	+ Iter 1	+ Iter 2	+ Iter 3	Final Rate
Tier 1	9/9	9/9	9/9	9/9	100.0%
Tier 2	8/8	8/8	8/8	8/8	100.0%
Tier 3	9/12	12/12	12/12	12/12	100.0%
Tier 4	2/9	2/9	6/9	7/9	77.8%
Total	28/38 (73.7%)	31/38 (81.6%)	35/38 (92.1%)	36/38 (94.7%)	94.7%

Table 4: Cumulative compilation results after iterative repair for Gemini 2.5 Pro. Two Tier 4 modules (fpu_dp_add, rob) remain non-compiling after three repair iterations.

threshold_buffer_fifo) both compile after Iter 2. The final compile rate reaches 100% (38/38), up from 73.7% one-shot. Notably, all four Tier 4 failures are resolved in a single iteration, because those errors were purely API-level (array indexing, native_decide misuse) and the compiler feedback directly identified the fix.

Gemini 2.5 Pro requires up to three iterations and does not fully converge (Table 4). Iter 1 resolves all three Tier 3 failures (9/12 $\rightarrow$ 12/12) but leaves all seven Tier 4 FPU and ROB modules non-compiling. Iter 2 recovers four of those seven, and Iter 3 recovers one more. Two Tier 4 modules (fpu_dp_add and rob) remain non-compiling after three iterations, likely because their errors require structural changes to the formalization (e.g., replacing propositional predicates with Boolean functions) that the model does not independently discover from compiler messages alone. The final compile rate for Gemini reaches 94.7% (36/38).

Table 5 tracks how theorem content changes through repair. One-shot theorem counts are measured on the original files regardless of compilation status, making them a proxy for how ambitiously each model attempts proof content even when generation fails. GPT-5.4 generated 7.5 and 8.0 theorems per file on average for its Tier 3 and Tier 4 one-shot failures respectively, and these counts are largely preserved through repair ($\Delta = 0.0$ for Tier 4; $\Delta = -1.0$ for Tier 3, where two files simplified proof content to resolve native_decide misuse). Gemini’s one-shot theorem counts for the same failing files are much lower (4.0 for Tier 3, 1.9 for Tier 4), and decrease further after repair ($\Delta = -0.3$ and $\Delta = -0.6$ respectively). This contrast suggests GPT-5.4 maintains a consistent proof-generation strategy across repair rounds, while Gemini trims proof content under repair pressure, particularly for the hardest Tier 4 designs.

7.4 Limitations of the current evaluation methods

A major limitation of the current evaluation is that our checks validate the generated Lean artifact as a Lean program, but do not yet establish semantic consistency between the Lean model and the original Verilog design. Compilation success shows that the artifact is accepted by Lean, schema conformance shows that it follows the intended target structure, and the sorry-free check rules out unfinished proof placeholders. However, none of these checks guarantee that the generated Lean model implements the same behavior as the source RTL. As a result, it is possible for a generated artifact to typecheck and pass the structural checks while still deviating semantically from the Verilog design, for example due to an incorrect bit-width, update priority, reset behavior, signedness interpretation, memory indexing convention, or state-transition rule.

Model	Tier	Repaired files	Theorem count			Final compiling
			One-shot	Final	Δ	
GPT-5.4	Tier 3	6	7.5	6.5	-1.0	6/6
	Tier 4	4	8.0	8.0	0.0	4/4
Gemini 2.5 Pro	Tier 3	3	4.0	3.7	-0.3	3/3
	Tier 4	7	1.9	1.3	-0.6	5/7

Table 5: Theorem count for modules sent to repair, restricted to one-shot failures. One-shot counts are taken directly from the non-compiling files. Final counts reflect the last repair iteration for each file, regardless of whether it ultimately compiles. A negative Δ indicates that the model trimmed proof content to resolve errors.

This limitation is especially important because the goal of Verilog to-Lean autoformalization is not only to produce valid Lean code, but to produce a faithful formal model of the original hardware. The current evaluation therefore measures syntactic validity, schema discipline, and limited usability, but it should not be interpreted as a full semantic correctness result. Lightweight representative checks can catch some obvious errors, but they are inherently incomplete and may miss corner cases that would be exposed only by exhaustive or formal comparison against the RTL.

One possible direction for addressing this limitation is to introduce a round-trip consistency check. Given a generated Lean model, a second model or translator could back-translate the Lean artifact into Verilog. The resulting Verilog design could then be compared against the original ground-truth Verilog using standard formal equivalence checking. For combinational modules, this could take the form of combinational equivalence checking over all inputs. For sequential modules, it would require a sequential equivalence check or a bounded equivalence check over the state-machine behavior, possibly together with an explicit mapping between the original RTL state and the generated Lean state. If the back-translated Verilog is equivalent to the original design, this would provide stronger evidence that the Lean model preserves the semantics of the source RTL.

8 Discussions and Implications

The results suggest that the most stable success of the current pipeline is not one-shot compilability, but structured formalization. Across both GPT-5.4 and Gemini 2.5 Pro, the generated artifacts consistently satisfy the target schemas even when some of them fail to compile. This indicates that the schema design is doing meaningful work as it constrains the translation problem enough for the model to recover the intended high-level organization of combinational, sequential, controller-style, and more advanced datapath modules. In that sense, the project shows that Verilog-to-Lean translation can be made substantially more structured than unconstrained code generation.

At the same time, the compilation results make clear that structural correctness alone is not sufficient. The main difficulty lies in producing Lean artifacts that are not only schema-shaped, but also executable and type-correct under the target Lean environment. This gap is most visible in Tier 3 and Tier 4, where the generated models must handle arrays, richer control logic, helper predicates, and more involved theorem statements. The observed failure modes therefore suggest that the dominant bottleneck is no longer broad misunderstanding of RTL structure, but the interaction between hardware-oriented modeling patterns and Lean’s executable semantics.

The iterative refinement results materially strengthen this conclusion. Under one-shot generation, both models compile 28 of 38 modules, but their strengths differ across tiers: GPT-5.4 performs better on Tier 4, while Gemini 2.5 Pro performs better on Tier 3. After checker-guided repair, however, GPT-5.4 reaches full compilation on all 38 modules, while Gemini 2.5 Pro reaches 36 of 38. This change is important because it shows that many initial failures are not fundamental translation breakdowns. Instead, they are often repairable mismatches between a largely reasonable formalization and the specific API, tactic, or typing requirements of Lean 4.29. In this respect, the checker is not merely an evaluation filter but becomes an active part of the generation loop.

The theorem-count results provide a similar qualitative signal. GPT-5.4 tends to preserve most of its proof content through repair, suggesting that it can often revise the surrounding Lean structure

without abandoning the intended theorem skeletons. Gemini, by contrast, tends to reduce theorem content during repair, especially on Tier 4 failures. This behavior suggests that different models do not only differ in final compile rate, but also in how they trade off proof ambition against compilability under repair pressure. That distinction may become increasingly important in later stages of the project, where theorem quality matters at least as much as model compilation.

The remaining unresolved cases are also informative. The two Tier 4 modules that remain non-compiling for Gemini after three repair rounds suggest that the hardest failures are not purely local syntax issues. Instead, some modules appear to require a deeper representational change, such as reformulating predicates as Boolean computations or restructuring the generated state-transition logic more substantially than compiler feedback alone can suggest. This points to an important next step for the project: stronger intermediate checkers that can diagnose semantic modeling issues earlier, rather than relying only on downstream compiler messages.

More broadly, these findings support a specific view of Lean-based hardware autoformalization. In the near term, the most realistic workflow is not fully autonomous theorem proving over arbitrary RTL. A more practical target is a staged pipeline in which schemas fix the representation, LLMs generate an initial Lean model, and machine-readable checks iteratively refine that model into a compilable artifact. Under this view, LLMs are most useful not as standalone formal verifiers, but as front-end formalization engines operating inside a tightly constrained verification loop. That is a narrower claim than full Lean-native hardware verification, but it is also a much more credible and empirically supported one at the current stage of the project.

9 Related Work

9.1 Theorem Proving for Hardware

Proof assistants have long been used for hardware verification, especially in settings where the goal is to reason about designs semantically rather than only through fixed property languages. Earlier lines of work in HOL and ACL2 demonstrated that machine-checked reasoning could scale to substantial hardware systems, including processor verification and industrial hardware flows [12, 13]. More recent work in Coq, including Coquet, Kami, and Silver Oak, further showed that modern proof assistants can support executable hardware models, modular specifications, and reusable correctness arguments [14, 8, 9]. Our work is aligned with this broader tradition, but focuses on a different bottleneck: not proving properties over a manually developed hardware model, but generating the Lean model from RTL in the first place.

9.2 Lean and LLM-Assisted Formal Reasoning

Lean has emerged as a widely used interactive theorem prover with a growing ecosystem for formal reasoning in mathematics and computer science [2–4]. Recent work such as LeanDojo, Lean Copilot, and AlphaProof suggests that large language models can assist formal reasoning by retrieving useful premises, generating tactics, or guiding proof search inside a proof assistant [5–7]. Our setting differs from these efforts in an important way. Rather than proving theorems over already formalized statements, we study *hardware autoformalization*: the problem of translating Verilog RTL into a compilable and structured Lean representation that can later serve as a basis for proof-oriented reasoning.

9.3 RTL Translation, Structured Front Ends, and the Remaining Gap

Our work is also related to structured RTL tooling such as Yosys and Surelog/UHDM, which provide parser- and IR-based access to Verilog and SystemVerilog designs [10, 11, 15]. These systems are valuable for extracting syntax and elaborated design structure, and they motivate possible hybrid approaches that combine deterministic RTL analysis with LLM-based formalization. In a related but different direction, prior work such as Fe-Si has studied verified compilation from high-level hardware descriptions into lower-level implementations [16]. By contrast, the problem we study runs in the opposite direction: starting from existing RTL and producing Lean-native formal models that follow fixed schemas and can be refined through compiler- and checker-guided feedback. To our knowledge, this RTL-to-Lean autoformalization setting remains relatively underexplored.

10 Conclusion

This project studies Verilog-to-Lean autoformalization as a first step toward Lean-based hardware verification. We define fixed target schemas for multiple classes of RTL modules, curate a 38-module benchmark spanning four tiers of design complexity, and evaluate LLM-based translation under both one-shot generation and checker-guided iterative repair. Across both GPT-5.4 and Gemini 2.5 Pro, the results show that schema-constrained prompting reliably produces structurally valid Lean artifacts, even when full compilation is not achieved in one shot.

The empirical results also show that iterative refinement substantially changes the practical picture. While both models compile 28 of 38 modules under one-shot generation, checker-guided repair raises GPT-5.4 to 38 of 38 compilable artifacts and Gemini 2.5 Pro to 36 of 38. These gains suggest that many initial failures are not fundamental breakdowns in RTL understanding, but repairable mismatches between generated formalizations and the specific executable requirements of Lean. At the same time, the remaining hard Tier 4 cases indicate that some modules still require deeper representational changes than compiler feedback alone can reliably induce.

Overall, our results support a pragmatic view of LLM-assisted hardware formalization. In the near term, the most credible use of LLMs is not as fully autonomous hardware theorem provers, but as front-end formalization engines operating within a constrained verification loop defined by fixed schemas, compiler checks, and repair feedback. Although the current evaluation does not yet establish full semantic equivalence between generated Lean models and their source RTL, it shows that Verilog-to-Lean autoformalization is already feasible in a structured and empirically meaningful form. We view this as a promising foundation for future work on stronger semantic checkers, hybrid parser-assisted pipelines, and eventually proof generation over automatically formalized hardware models.

References

- [1] {Thomas A.} Henzinger and Helmut Veith. *Handbook of Model Checking*. Springer, United States, 2018. ISBN 978-3-319-10574-1. doi: 10.1007/978-3-319-10575-8.
- [2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science (LNAI)*, pages 625–635. Springer, Cham, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.
- [3] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020*, New Orleans, LA, USA, January 2020. ACM. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.
- [4] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The Lean computer science library, 2026. URL <https://arxiv.org/abs/2602.04846>.
- [5] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. Leandjo: Theorem proving with retrieval-augmented language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 21573–21612. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/4441469427094f8873d0fecb0c4e1cee-Paper-Datasets_and_Benchmarks.pdf.
- [6] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean copilot: Large language models as copilots for theorem proving in lean, 2025. URL <https://arxiv.org/abs/2404.12534>.
- [7] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sonne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. URL <https://doi.org/10.1038/s41586-025-09833-y>.
- [8] Joonwon Choi, Muralidaran Vijayaraghavan, Benjamin Sherman, Adam Chlipala, and Arvind. Kami: a platform for high-level parametric hardware specification and its modular verification. *Proc. ACM Program. Lang.*, 1(ICFP), August 2017. doi: 10.1145/3110268. URL <https://doi.org/10.1145/3110268>.
- [9] Project Oak. Silveroak: Formal specification and hardware generators. <https://github.com/project-oak/silveroak>, 2022.
- [10] Clifford Wolf, Johann Glaser, and Johannes Kepler. Yosys-a free verilog synthesis suite. In *Proceedings of the Austrian Workshop on Microelectronics (Austrochip)*, 2013.
- [11] CHIPS Alliance and contributors. Surelog: Systemverilog 2017 pre-processor, parser, elaborator, uhdh compiler. <https://github.com/chipsalliance/Surelog>, 2026. Open-source SystemVerilog 2017 front-end producing UHDM.
- [12] Michael J. C. Gordon. *HOL: A Proof Generating System for Higher-Order Logic*, pages 73–128. Springer US, Boston, MA, 1988. ISBN 978-1-4613-2007-4. doi: 10.1007/978-1-4613-2007-4_3. URL https://doi.org/10.1007/978-1-4613-2007-4_3.
- [13] Jr Hunt, Warren A., Matt Kaufmann, J Strother Moore, and Anna Slobodova. Industrial hardware and software verification with acl2. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 375(2104):20150399, 09 2017. ISSN 1364-503X. doi: 10.1098/rsta.2015.0399. URL <https://doi.org/10.1098/rsta.2015.0399>.
- [14] Thomas Braibant. Coquet: a coq library for verifying hardware, 2011. URL <https://arxiv.org/abs/1108.4253>.
- [15] Alain Dargelas and Henner Zeller. Universal hardware data model. In *Proceedings of the Workshop on Open-Source EDA Technology (WOSET)*, October 2020. URL <https://woset-workshop.github.io/PDFs/2020/a10.pdf>.
- [16] Thomas Braibant and Adam Chlipala. Formal verification of hardware synthesis. In *Proceedings of the 25th International Conference on Computer Aided Verification - Volume 8044, CAV 2013*, page 213–228, Berlin, Heidelberg, 2013. Springer-Verlag. ISBN 9783642397981.

A Appendix

A.1 Schemas for Each Design

A.1.1 Schema A

Schema A: Combinational Modules (Tier 1)

Combinational modules have no state — they are pure functions from inputs to outputs.

Required structure

```
namespace <ModuleName>_<VariantId>

-- Type abbreviations (use BitVec for multi-bit, Bool for single-bit)
abbrev Bit := Bool
abbrev BitsN := BitVec N          -- e.g., Bits4 := BitVec 4

-- 1. Input structure: one field per module input port
structure Inputs where
  <field> : <type>
  ...
deriving Repr, DecidableEq

-- 2. Output structure: one field per module output port
structure Outputs where
  <field> : <type>
  ...
deriving Repr, DecidableEq

-- 3. (Optional) Helper constructors for BitVec literals
def bvN (n : Nat) : BitsN := BitVec.ofNat N n

-- 4. Evaluation function: the combinational logic
def eval (input : Inputs) : Outputs := ...

-- 5. Theorems: concrete test-vector checks or symbolic properties
--   Use `native_decide` for concrete test vectors
--   Use `rfl` when the proof is definitional
theorem <name> : eval { ... } = { ... } := by native_decide

end <ModuleName>_<VariantId>
```

Rules

- One namespace per module variant
- All structures must derive Repr and DecidableEq
- The evaluation function must be named eval
- No sorry allowed
- Theorems should cover representative input cases (at minimum one per branch/case)
- Use BitVec for multi-bit signals, Bool for single-bit signals

A.1.2 Schema B

Schema B: Sequential Modules (Tier 2+)

Sequential modules have state that evolves over clock cycles.

Required structure

```
import Std
```

```

namespace <ModuleName>

-- Helper definitions (e.g., wrapping arithmetic)
def modulus (w : Nat) : Nat := 2 ^ w
def wrap (w : Nat) (n : Nat) : Nat := n % modulus w

-- 1. State structure: internal registers / flip-flop values
structure State (w : Nat) where -- parameterize by width if applicable
  <field> : <type>
  ...
deriving Repr, DecidableEq

-- 2. Input structure: all input ports
structure Input where
  rst : Bool
  en : Bool
  ...
deriving Repr, DecidableEq

-- 3. Output structure: all output ports
structure Output where
  <field> : <type>
  ...
deriving Repr, DecidableEq

-- 4. Initial/reset state
def init (w : Nat) : State w := { ... } -- or `zeroState`

-- 5. Transition function: one clock-cycle state update
-- Must respect Verilog priority (e.g., rst > load > en > hold)
def step {w : Nat} (s : State w) (i : Input) : State w := ...

-- 6. Output function: combinational read of state
def out {w : Nat} (s : State w) : Output := ...

-- 7. Theorems with @[simp] attribute
-- Categories:
-- a) Reset behavior: step under rst = init
-- b) Hold behavior: step with nothing enabled = identity
-- c) Functional behavior: step under each operation
-- d) Priority: rst beats load beats en, etc.
-- e) Composition: multi-step properties (optional)
-- f) Output theorems: out step under various inputs

@[simp] theorem step_reset ... := by simp [step, init]
@[simp] theorem step_hold ... := by simp [step]
@[simp] theorem step_op ... := by simp [step, ...]

end <ModuleName>

```

Rules

- State must capture all flip-flop / register values from the Verilog module
- step must faithfully encode the priority of the Verilog always block
- init / zeroState must match the Verilog reset behavior
- out is a pure function from state to outputs (Moore-style)
- Parameterize by width w when the Verilog module uses parameter
- Use Nat with wrapping ($\% 2^w$) or BitVec w for fixed-width arithmetic
- No sorry allowed

- Theorems should cover: reset, hold, each operation, and priority ordering

A.1.3 Schema C

Schema C: Controllers / FSMs (Tier 3+)

Extends Schema B with additional requirements for complex controllers:

- **FSM state encoding:** Use an inductive type for named states
- **Multiple state variables:** State structure may have multiple fields (e.g., pointer, count, memory)
- **Richer properties:** Mutual exclusion of grants, FIFO ordering, fairness
- **Memory modeling:** Use Array or Vector for register files / buffers

```
-- Example: FSM states as an inductive type
inductive FsmState where
  | idle | active | done
deriving Repr, DecidableEq
```

```
structure State where
  phase : FsmState
  counter : Nat
  ...
```

Common conventions

Verilog Construct	Lean Representation
wire / reg (1-bit)	Bool
[N-1:0] bus	BitVec N or wrapping Nat
always @(posedge clk)	step function
always @(*)	eval function for combinational logic
parameter WIDTH = N	Type parameter (w : Nat)
if/else priority chain	Nested if/then/else preserving priority order
case statement	match expression
Reset to 0	init / zeroState with zero-valued fields

A.2 Prompts for Experiments

A.2.1 One-shot Prompt

One-shot Prompt

Prompt A — One-Shot Translation (Sys A)

You are an expert in both Verilog RTL design and the Lean 4 theorem prover.

Your task is to translate the given Verilog module into a Lean 4 formal model following the exact schema described below.

=== LEAN 4 TARGET SCHEMA ===

For COMBINATIONAL modules (no clocked always block):

```
namespace <ModuleName>_01
```

```

-- Type abbreviations:
  use Bool for 1-bit signals,
  BitVec N for N-bit signals
abbrev Bit := Bool
abbrev BitsN := BitVec N

structure Inputs where
  <field> : <type>
  ...
deriving Repr, DecidableEq

structure Outputs where
  <field> : <type>
  ...
deriving Repr, DecidableEq

-- Optional: helper constructors for BitVec literals
def bvN (n : Nat) : BitsN := BitVec.ofNat N n

-- Evaluation function (required, must be named `eval`)
def eval (input : Inputs) : Outputs := ...

-- Theorems: at least one per branch/case, use native_decide or rfl
theorem <name> : eval { ... } = { ... } := by native_decide

end <ModuleName>_01

For SEQUENTIAL modules (has clocked always @(posedge clk) block):

import Std

namespace <ModuleName>

structure State ... deriving Repr, DecidableEq -- internal registers
structure Input ... deriving Repr, DecidableEq -- input ports
  (no clk/rst in struct if rst is handled in step)
structure Output ... deriving Repr, DecidableEq -- output ports

def init : State := ... -- reset state

-- Transition function: encode the Verilog
always block priority exactly
(e.g., if rst has highest priority, check it first)
def step (s : State) (i : Input) : State := ...

-- Output function: pure function from state to outputs
def out (s : State) : Output := ...

-- Theorems with @[simp]:
-- Cover reset, hold (no enable), each operation, priority ordering
@[simp] theorem step_reset ... := by simp [step, init]
@[simp] theorem step_hold ... := by simp [step]
...

end <ModuleName>

=== REQUIREMENTS ===
- Do NOT use `sorry` anywhere.
- Only use API available for Lean 4.29

```

- All structures must derive ``Repr`` and ``DecidableEq``.
- The namespace must match the Verilog module name.
- Faithfully encode Verilog semantics (bit widths, priority of if/else, reset value).
- For multi-bit signals, use `BitVec` with the correct width.
- For wrap-around counters, use modular arithmetic $(\% 2^w)$.
- Theorems must actually compile - do not write theorems you are not confident about. If unsure, write fewer theorems rather than using `sorry`.
- Do not add `clk` to the Input structure; model clocked behavior with the `step` function.

=== VERILOG MODULE TO TRANSLATE ===

{{VERILOG}}

=== OUTPUT ===

Produce only the Lean 4 code. No explanation, no markdown fences, just the raw Lean source.

A.2.2 Iterative Refinement Prompt

Iterative Refinement Prompt

Prompt D — Iterative Repair (Sys D)

You are an expert in both Verilog RTL design and the Lean 4 theorem prover.

The Lean 4 translation below was generated from the Verilog module, but it contains errors. Your task is to fix all errors and produce a corrected Lean file.

=== ORIGINAL VERILOG ===

{{VERILOG}}

=== PREVIOUS LEAN TRANSLATION (contains errors) ===

{{LEAN_WITH_ERRORS}}

=== COMPILER / CHECKER ERRORS ===

{{ERROR_OUTPUT}}

=== INSTRUCTIONS ===

1. Fix all reported errors.
2. Do NOT introduce ``sorry`` to suppress errors - fix them properly.
3. Keep the same overall structure (namespace, State/Input/Output structs, step/eval functions, theorems). Only change what is necessary to fix errors.
4. If a theorem cannot be proved, DELETE the theorem entirely rather than using `sorry`.
5. Preserve correct parts of the translation unchanged.
6. Make sure all structures still derive ``Repr`` and ``DecidableEq``.

=== OUTPUT ===

Produce only the corrected Lean 4 code. No explanation,

no markdown fences, just the raw Lean source.